

TrackB Training Local Test Notebook

Overview

This notebook provides a complete Track B training pipeline for the Ego4D Short-Term Anticipation (STA) task. It supports both ResNet18 and VideoMAE backbones with multi-task learning, mixed precision training (AMP), and comprehensive evaluation metrics.

Notebook Structure

Part	Cells	Description
Part 1	1-4	Environment Setup
Part 2	5-7	Configuration
Part 3	8-12	Data Validation
Part 4	13-15	Tokenization Pipeline
Part 5	16-19	Dataset Creation
Part 6	20-22	Model Architecture
Part 7	23-27	Training Loop
Part 8	28-29	Visualization
Part 9	30-33	Evaluation & Inference
Part 9.5	34-36	Advanced Evaluation (Hotspot + CLIP + STA Metrics)
Part 10	37-38	Summary
Part 11	39-41	Backbone Comparison

Detailed Cell-by-Cell Documentation

Part 1: Environment Setup

Cell 3 - Device Detection

Description	
Inputs	None
Outputs	<code>DEVICE</code> (str): "cuda" or "cpu"
Purpose	Detect available compute device

Cell 4 - Path Setup

Description	
Inputs	None
Outputs	LOCAL_EXT (Path), REPO_ROOT (Path), PATHS (dict)
Purpose	Configure all directory paths for data, models, and outputs

PATHS dictionary:

```
{
  'frames': 'local_extraction/v2/extracted_frames',
  'manifests': 'local_extraction/v2/manifests',
  'trackA_runs': 'local_extraction/runs/Track_A',
  'trackB_runs': 'local_extraction/runs/Track_B',
  'configs': 'local_extraction/configs',
  'annotations': 'local_extraction/v2/org_annotations',
}
```

Part 2: Configuration

Cell 6 - Backbone Selection

Description	
Inputs	User choice: BACKBONE = "resnet18" or "videomae"
Outputs	BACKBONE (str), BACKBONE_SPECS (dict), spec (dict)
Purpose	Select backbone and get its specifications

Backbone Specifications:

Backbone	Dimension	Grid	Tokens	Frames
ResNet18	512	7×7	49	8
VideoMAE	768	14×14	196	16

Cell 7 - Training Configuration

Description	
Inputs	BACKBONE , DEMO_MODE toggle
Outputs	cfg (Config dataclass)
Purpose	Set all training hyperparameters

Key Config Fields:

```
@dataclass
class Config:
    backbone: str          # "resnet18" or "videomae"
    token_dim: int = 256   # Projector output dimension
    epochs: int            # 2 (demo) or 20 (full)
    batch_size: int        # 2 (demo) or 4 (full)
    lr: float = 1e-3
    use_amp: bool = False  # Mixed Precision
    use_multi_task: bool = True
    loss_w_next: float = 1.5
    loss_w_noun: float = 0.25
    loss_w_verb: float = 0.25
    loss_w_ttc: float = 0.1
```

Part 3: Data Validation

Cell 10 - Import Track B Modules

Description	
Inputs	Python path configured
Outputs	Imported modules from trackB/
Purpose	Import tokenizer, dataset, fusion, head, metrics

Imported Components:

- trackB_tokenizer : TokenizerConfig, build_backbone, roi_pool_tokens_mean
- trackB_dataset : TrackBDataset, trackB_collate
- trackB_fusion : FusionConfig, TrackBFusion
- trackB_head : HeadConfig, TrackBHead
- trackB_metrics : binary_accuracy, binary_average_precision, ttc_mae

Cell 11 - Find Training Manifests

Description	
Inputs	STAGEB_RUN_NAME , PATHS['trackA_runs']
Outputs	stageB_run (Path), train_manifest (Path), val_manifest (Path), train_records (list)
Purpose	Locate Stage B manifests from Track A run

Expected Manifest Format (JSONL):

```
{"uid": "video_uid", "frame_idx": 1234, "candidates": [...], ...}
```

Cell 12 - Validate Frame Coverage

Description	
Inputs	<code>train_records</code> , <code>PATHS['frames']</code>
Outputs	<code>manifest_uids</code> (set), <code>frame_uids</code> (set), <code>covered</code> (set), <code>missing</code> (set)
Purpose	Check that extracted frames exist for all manifest UIDs

Part 4: Tokenization Pipeline

Cell 14 - Build Tokenizer

Description	
Inputs	<code>cfg.backbone</code> , <code>cfg.device</code>
Outputs	<code>tok_cfg</code> (TokenizerConfig), <code>backbone</code> (nn.Module), <code>transform</code> (Compose)
Purpose	Initialize backbone model and image transforms

TokenizerConfig:

```
tok_cfg.video_backbone = "resnet18" # or "videomae"  
tok_cfg.num_frames = 8 # or 16 for videomae  
tok_cfg.device = "cuda"
```

Cell 15 - Test Tokenization

Description	
Inputs	Sample frame path, <code>backbone</code> , <code>transform</code> , <code>tok_cfg</code>
Outputs	<code>img_tokens</code> (Tensor: N×D), <code>vid_tokens</code> (Tensor: T×N×D), <code>hw</code> (tuple)
Purpose	Verify tokenization on a sample frame

Output Shapes:

- Image tokens: `(49, 512)` for ResNet18, `(196, 768)` for VideoMAE
- Video tokens: `(T, 49, 512)` or `(T, 196, 768)`

Part 5: Dataset Creation

Cell 17 - Create Training Dataset

Description	
Inputs	<code>PATHS['frames']</code> , <code>stageB_run</code> , <code>train_manifest</code> , <code>tok_cfg</code> , <code>cfg.candidate_limit</code>
Outputs	<code>train_ds</code> (TrackBDataset)
Purpose	Create PyTorch dataset for training

Dataset Item Structure:

```
{
  'uid': str,
  'frame_path': Path,
  'img_tokens': Tensor(N, D),
  'vid_tokens': Tensor(T, N, D),
  'bboxes': List[Tuple[x1, y1, x2, y2]],
  'is_positive': Tensor(num_candidates),
  'ttc': Tensor(num_candidates),
  'gt_noun_id': Tensor(num_candidates),
  'gt_verb_id': Tensor(num_candidates),
  'hw': Tuple[H, W],
  'image_size': Tuple[W, H],
}
```

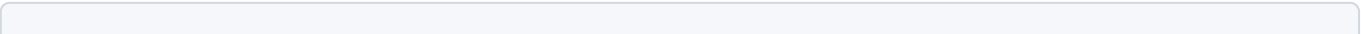
Cell 18 - Create Validation Dataset

Description	
Inputs	<code>val_manifest</code> , same params as training
Outputs	<code>val_ds</code> (TrackBDataset)
Purpose	Create validation dataset

Cell 19 - Test DataLoader

Description	
Inputs	<code>train_ds</code> , <code>val_ds</code> , <code>cfg.batch_size</code>
Outputs	<code>train_loader</code> (DataLoader), <code>val_loader</code> (DataLoader)
Purpose	Create data loaders with custom collate function

Batch Structure:



```
{
    'valid': bool,
    'samples': List[dict], # List of dataset items
}
```

Part 6: Model Architecture

Cell 21 - Build Model Components

Description	
Inputs	<code>spec['dim']</code> , <code>cfg.token_dim</code> , <code>cfg.fusion_layers</code> , <code>cfg.num_classes</code>
Outputs	<code>projector</code> (nn.Linear), <code>fusion</code> (TrackBFusion), <code>head</code> (TrackBHead)
Purpose	Create the three model components

Model Pipeline:

Backbone Tokens (D_in) → Projector (D_out=256) → Fusion (FGTP + Cross-Attn) → Head (Cls + TTC)

Component Shapes:

Component	Input	Output
Projector	(N, 512/768)	(N, 256)
Fusion	img:(1,N,256), vid:(1,T,N,256)	fused_img:(1,N,256), fused_vid:(1,N,256)
Head	fused_img, fused_vid	cls_logits:(1,N,K), ttc:(1,N,1)

Cell 22 - Test Forward Pass

Description	
Inputs	Sample batch, <code>projector</code> , <code>fusion</code> , <code>head</code>
Outputs	<code>outputs</code> (dict with cls_logits, ttc, optional noun_logits, verb_logits)
Purpose	Verify end-to-end forward pass

Part 7: Training Loop

Cell 24 - Training Setup

Description	
Inputs	Model parameters, <code>cfg.lr</code>
Outputs	<code>optimizer</code> (AdamW), <code>ce_loss</code> (CrossEntropyLoss), <code>mse_loss</code> (MSELoss), <code>scaler</code> (GradScaler if AMP), <code>run_dir</code> , <code>ckpt_dir</code>
Purpose	Initialize optimizer, loss functions, and output directories

Also defines:

- `top_k_accuracy()` function
- `top5_accuracy()` function
- `get_lr()` learning rate scheduler

Cell 25 - Training Function

Description	
Inputs	<code>epoch</code> , data loaders, models, optimizer
Outputs	<code>train_metrics</code> (dict)
Purpose	Train one epoch with multi-task losses and optional AMP

Multi-Task Loss Computation:

Total Loss = $w_{\text{next}} \times L_{\text{next}} + w_{\text{noun}} \times L_{\text{noun}} + w_{\text{verb}} \times L_{\text{verb}} + w_{\text{ttc}} \times L_{\text{ttc}}$

Returned Metrics:

```
{
    'loss': float,
    'loss_next': float,
    'loss_noun': float,
    'loss_verb': float,
    'loss_ttc': float,
    'accuracy': float,
    'mAP': float,
    'lr': float,
}
```

Cell 26 - Validation Function

Description

Description	
Inputs	<code>val_ds</code> , <code>val_loader</code> , <code>models</code>
Outputs	<code>val_metrics</code> (dict)
Purpose	Evaluate on validation set

Returned Metrics:

```
{
  'accuracy': float,
  'mAP': float,
  'top1_accuracy': float,
  'top5_accuracy': float,
  'ttc_mae': float,
}
```

Cell 27 - Full Training Loop

Description	
Inputs	All previous components
Outputs	<code>history</code> (dict), saved checkpoints, <code>RESULTS_FILE</code>
Purpose	Run complete training with early stopping

Saved Checkpoint Contents:

```
{
  'epoch': int,
  'projector': state_dict,
  'fusion': state_dict,
  'head': state_dict,
  'optimizer': state_dict,
  'scaler': state_dict (if AMP),
  'metrics': dict,
  'config': dict,
}
```

Output Files:

- `runs/Track_B/run_{backbone}_{timestamp}/checkpoints/best.pt`
- `runs/Track_B/run_{backbone}_{timestamp}/checkpoints/epoch_*.pt`
- `runs/Track_B/backbone_comparison.json`

Part 8: Visualization

Cell 29 - Training Curves

Description	
Inputs	<code>history</code>
Outputs	<code>training_curves.png</code>
Purpose	Plot loss, accuracy, mAP, and learning rate curves

Part 9: Evaluation & Inference

Cell 31 - Load Best Checkpoint

Description	
Inputs	<code>ckpt_dir / 'best.pt'</code>
Outputs	Updated model weights
Purpose	Load best checkpoint for evaluation

Cell 32 - Final Evaluation

Description	
Inputs	<code>val_loader</code> , models
Outputs	Final metrics printed
Purpose	Run final evaluation on validation set

Cell 33 - Visualize Predictions

Description	
Inputs	Sample validation frames, models
Outputs	<code>sample_predictions.png</code>
Purpose	Draw bounding boxes with predictions on sample frames

Part 9.5: Advanced Evaluation

Cell 35 - Hotspot Priors Configuration

Description	
Inputs	Configuration settings

Description	
Outputs	HOTSPOT_CONFIG , CLIP_CONFIG
Purpose	Configure PEAR-style hotspot priors and CLIP re-ranking

Hotspot Prior Formula:

$$\text{final_score} = (1 - \alpha) \times \text{model_score} + \alpha \times P(\text{next-active} \mid \text{noun, verb})$$

Cell 36 - STA Metrics (N/N+V/N+δ/All) with Top-1 and Top-5

Description	
Inputs	val_ds , val_loader , models, iou_thresh=0.5
Outputs	sta_metrics (dict)
Purpose	Compute Ego4D STA challenge metrics

STA Metrics:

Metric	Description	Top-1	Top-5
N	Noun correct	✓	✓
N+V	Noun + Verb correct	✓	✓
N+δ	Noun + TTC bin correct	✓	✓
All	Noun + Verb + TTC correct	✓	✓

Returned Metrics:

```
{
  'N_top1_acc': float,
  'N_top5_acc': float,
  'NV_top1_acc': float,
  'NV_top5_acc': float,
  'Nd_top1_acc': float,
  'Nd_top5_acc': float,
  'All_top1_acc': float,
  'All_top5_acc': float,
}
```

Cell 38 - Training Summary

Description	
Inputs	<code>history</code> , <code>cfg</code> , <code>run_dir</code>
Outputs	Summary printed to console
Purpose	Display final training summary and next steps

Part 11: Backbone Comparison

Cell 40 - Load and Compare Results

Description	
Inputs	<code>RESULTS_FILE</code> (<code>backbone_comparison.json</code>)
Outputs	Comparison table printed
Purpose	Compare ResNet18 vs VideoMAE results

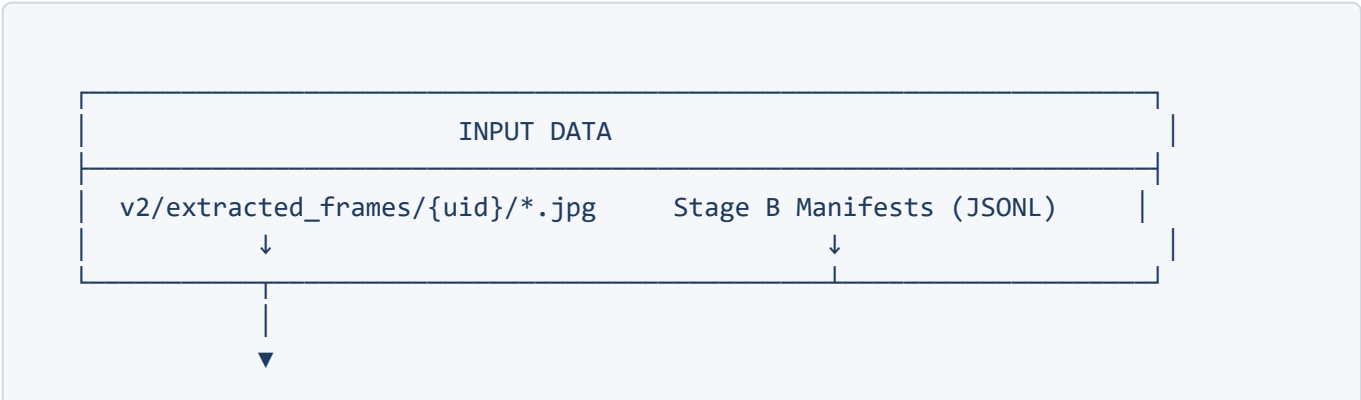
Comparison Table Columns:

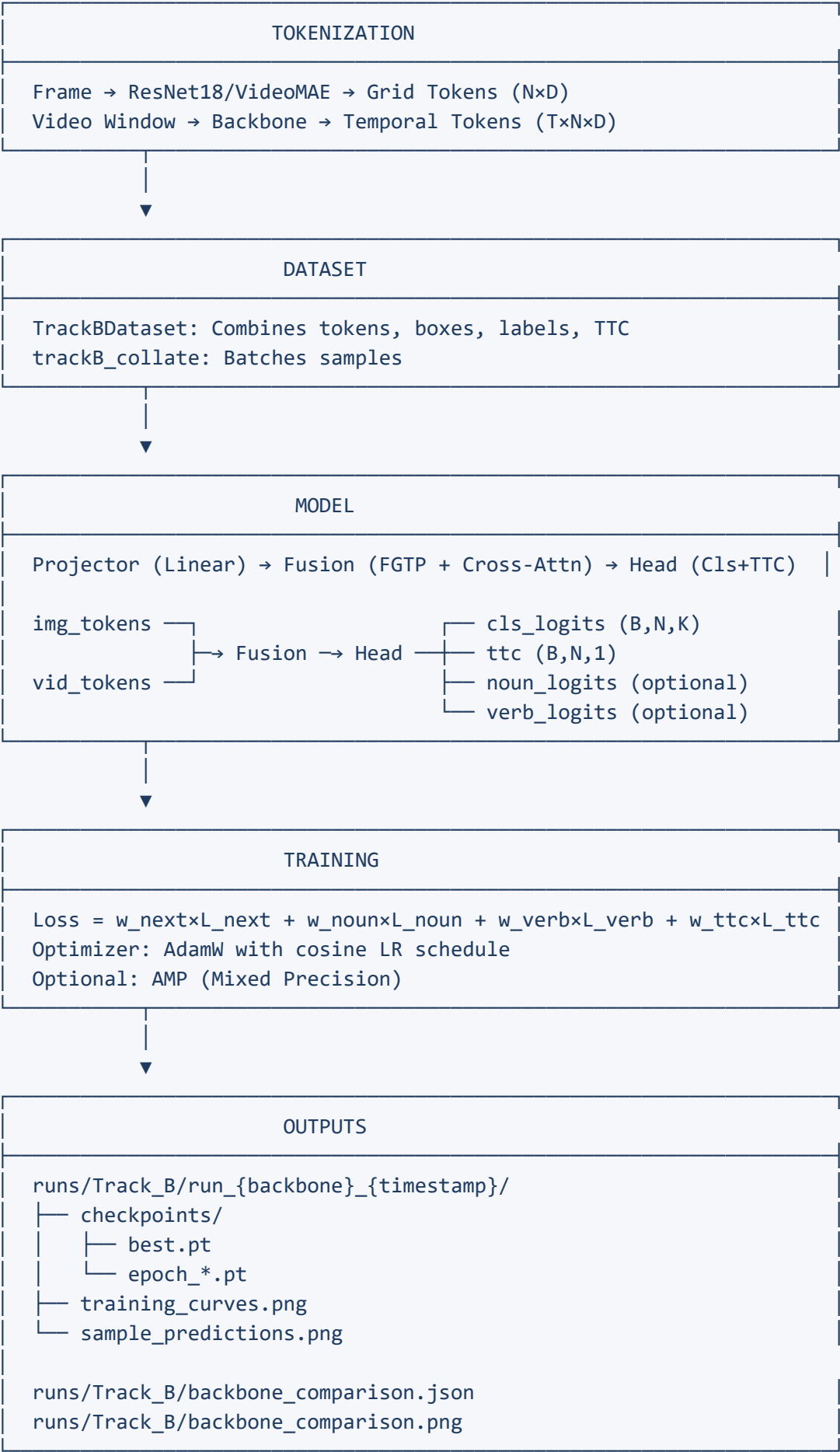
- Backbone
- Val Accuracy
- Val mAP
- Val Top-5
- TTC MAE

Cell 41 - Visualize Backbone Comparison

Description	
Inputs	<code>RESULTS_FILE</code>
Outputs	<code>backbone_comparison.png</code>
Purpose	Bar chart comparing backbone performance

Data Flow Diagram





Quick Start

```
# 1. Set your backbone
BACKBONE = "resnet18" # or "videomae"

# 2. Set demo mode for quick test
DEMO_MODE = True # False for full training

# 3. Run all cells in order

# 4. To compare backbones:
#   - Run with BACKBONE = "resnet18"
#   - Run with BACKBONE = "videomae"
#   - Run Part 11 comparison cells
```

Requirements

- PyTorch >= 1.9
- torchvision
- tqdm
- Pillow
- matplotlib
- (Optional) CLIP for re-ranking

TrackB Modules Used

Module	Purpose
trackB_tokenizer.py	Backbone + tokenization
trackB_dataset.py	Dataset + collate
trackB_fusion.py	FGTP + Cross-Attention
trackB_head.py	Classification + TTC heads
trackB_metrics.py	Evaluation metrics